实验报告

一、实验概述

本次实验选择“云原生计数器”作为应用场景。该应用采用 Flask 编写后端，提供 HTML 页面与 JSON API 两种访问方式，使用 Redis 作为轻量级数据存储保存访问计数。系统包含 api 与 redis 两个可区分的运行单元，其中 api 以 2 副本部署，用于验证服务发现与负载均衡。

技术栈包括 Docker、docker-compose、Docker Desktop 内置 Kubernetes（kind 驱动）、Flask 和 Redis。实验目标为：完成应用容器化、K8s 部署、ConfigMap/Secret 配置注入、自愈验证、扩缩容验证，并对比 Docker 与 K8s 在部署与管理上的差异。

实验最初尝试使用 Minikube 搭建本地集群，但因国内网络无法访问 gcr.io 与 registry.k8s.io，多次重试失败，最终改用 Docker Desktop 内置 Kubernetes。该集群与 Docker 共享镜像存储，无需额外加载镜像，部署流程更简洁。本次实验的 K8s 验证均在该集群上完成。

实验代码与配置已上传至 GitHub: https://github.com/WRPSSKMT/cloud-native-lab

二、环境与工具

项目	实际环境
宿主机系统	Windows 11 家庭中文版 25H2
CPU / 内存	32 核 / 7.28 GiB（Docker Desktop 可用）
虚拟化方案	Docker Desktop + WSL2
WSL2 内核	6.18.54.1-microsoft-standard-WSL2
Docker 版本	29.8.2
Docker Compose 版本	v5.6.0
Kubernetes 版本	v1.36.1
集群工具	Docker Desktop 内置 Kubernetes（kind 驱动，1 节点）
应用技术栈	Python 3.11 + Flask + Redis 7 (alpine)
镜像基础	python:3.11-slim

```
(base) PS C:\windows\system32> wsl --install
正在下载: 适用于 Linux 的 Windows 子系统 3.0.3
正在安装: 适用于 Linux 的 Windows 子系统 3.0.3
已安装 适用于 Linux 的 Windows 子系统 3.0.3。
操作成功完成。
正在下载: Ubuntu
服务器返回无效或不可识别的响应
错误代码:  Wsl/InstallDistro/WININET_E_INVALID_SERVER_RESPONSE
如果此错误是意外错误，请考虑搜索现有问题或在 https://github.com/microsoft/WSL/issues 提交新问题。
(base) PS C:\windows\system32> wsl --install -d Ubuntu
正在下载: Ubuntu
正在安装: Ubuntu
已成功安装分发。可以通过“wsl.exe -d Ubuntu”启动它
正在启动 Ubuntu...
Provisioning the new WSL instance Ubuntu
This might take a while...
Create a default Unix user account: w
New password:
Retype new password:
passwd: password updated successfully
usermod: no changes
Help improve Ubuntu!

Help us improve Ubuntu features and compatibility by sharing system reports with Canonical.
Reports are sent anonymously and do not contain any personal data.
For legal details, please visit: https://ubuntu.com/legal/systems-information-notice

We will save your answer to Windows and will only ask you once.

Would you like to opt-in to platform metrics collection (Y/n)? To see an example of the data collected, enter 'e'.
[Y/n/e]: n
w@LAPTOP-LG6ORFK5:/mnt/c/windows/system32$ exit
```

```
(base) PS C:\Users\W> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
(base) PS C:\Users\W> docker version
Client:
 Version:           29.8.2
 API version:       1.56
 Go version:        go1.26.8
 Git commit:        7fc2dff
 Built:             Wed Sep 30 19:35:42 2026
 OS/Arch:           windows/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.95.0 (242831)
 Engine:
  Version:          29.8.2
  API version:      1.56 (minimum version 1.40)
  Go version:       go1.26.8
  Git commit:       8af9fe3
  Built:            Wed Sep 30 19:32:25 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.3.6
  GitCommit:        ee2735368117d2eb259779949d5e75cdafec9761
 runc:
  Version:          1.5.1
  GitCommit:        v1.5.1-0-g8f2685a4
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

docker desktop 安装与验证

三、应用设计与实现

3.1 架构说明

组件	职责	关键实现
Flask Web	提供 HTTP 接口、返回实例标识	/、/api、/health
Redis	保存访问计数	INCR visits
Docker Compose	本地组合启动多个容器	api、redis 两个服务
Kubernetes Deployment	声明并维护应用副本	api 初始 2 副本
Kubernetes Service	提供稳定服务入口与负载分发	api、redis
ConfigMap / Secret	注入配置	Redis 地址、端口、密码

应用由 Flask Web 服务和 Redis 数据服务构成。客户端通过 HTTP 访问 Flask; Flask 将计数递增操作交给 Redis 保存。Docker Compose 通过服务名 redis 建立组件间通信; 在 Kubernetes 中，Redis Service 为数据服务提供集群内稳定 DNS 名称，api Service 将请求转发至带有 app=api 标签的应用 Pod。

3.2 核心接口

接口	功能	验证方法
GET /	返回 HTML 页面，显示 Pod 名与访问次数	curl http://localhost:8080/
GET /api	返回 JSON，含 pod 与 visits 字段	curl http://localhost:8080/api
GET /health	检查服务健康状态	curl http://localhost:8080/health

3.3 容器化思路

Dockerfile 基于 python:3.11-slim，安装 flask 与 redis 依赖，复制应用代码，暴露 5000 端口。配置通过环境变量注入，镜像本身不包含任何环境相关配置。

3.4 关键源码说明

app.py 中通过 os.getenv 读取 REDIS_HOST、REDIS_PORT、REDIS_PASSWORD，socket.gethostname() 返回当前 Pod 名，用于验证多副本负载均衡。

四、部署过程记录

4.1 项目文件结构

```
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> Get-ChildItem -Recurse -File | Select-Object FullName

FullName
--------
C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验\README.md
C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验\report.docx
C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验\docker\docker-compose.yml
C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验\docker\Dockerfile
C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验\k8s\all.yaml
C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验\src\app.py
C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验\src\requirements.txt
```

项目文件结构，包含 src、docker、k8s、screenshots 四个目录

4.2 Docker 镜像构建

```
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> docker build -t myapp:1.0 -f docker/Dockerfile .
[+] Building 518.7s (10/10) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                  0.0s
 => => transferring dockerfile: 209B                                                                                  0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                                                   6.2s
 => [internal] load .dockerignore                                                                                     0.0s
 => => transferring context: 2B                                                                                       0.0s
 => [1/5] FROM docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767a      432.4s
 => => resolve docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767af9     0.0s
 => => sha256:496c1a902b3a93cea3e54b90b79dbcaa0e58ad90c3fe9dd0a43c9557dc47f17d 250B / 250B                            3.0s
 => => sha256:759a7f30c31818dda4c97b407b24d93da479c306568d185ced927e1a749164f1 14.46MB / 14.46MB                     53.0s
 => => sha256:0f752a1982d6f819fc2c9e78ac1514c63f2f2e1dec7885ffd4cb6809ba4a26ca 1.29MB / 1.29MB                       10.3s
 => => sha256:ecc510c1e359bdc007b39802570bb7e4dec7d6ebccf357c3201a74299113e05e 29.84MB / 29.84MB                     431.0s
 => => extracting sha256:ecc510c1e359bdc007b39802570bb7e4dec7d6ebccf357c3201a74299113e05e                             0.7s
 => => extracting sha256:0f752a1982d6f819fc2c9e78ac1514c63f2f2e1dec7885ffd4cb6809ba4a26ca                             0.1s
 => => extracting sha256:759a7f30c31818dda4c97b407b24d93da479c306568d185ced927e1a749164f1                             0.5s
 => => extracting sha256:496c1a902b3a93cea3e54b90b79dbcaa0e58ad90c3fe9dd0a43c9557dc47f17d                             0.0s
 => [internal] load build context                                                                                     0.0s
 => => transferring context: 813B                                                                                     0.0s
 => [2/5] WORKDIR /app                                                                                                0.2s
 => [3/5] COPY src/requirements.txt .                                                                                 0.0s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                         77.4s
 => [5/5] COPY src/app.py .                                                                                           0.0s
 => exporting to image                                                                                                2.3s
 => => exporting layers                                                                                               0.6s
 => => exporting manifest sha256:e3cc843a1830fe179fc3a1393d7e7cdcc4b496d7cedd4fa11584955629a822b4                     0.0s
 => => exporting config sha256:27f7656f3d1d5435ae8980305dd591386616f34019eecefc22971628a4c01d6c                       0.0s
 => => exporting attestation manifest sha256:c672879db5b9b6ce4203453cd2960a9ca419759170d53dfa4e570fedbe6702d5        0.0s
 => => exporting manifest list sha256:85443dd1e46b54913cecb2abba41132410a5ceb9cf08f06d3d2478b850568ff6               0.0s
 => => naming to docker.io/library/myapp:1.0                                                                          0.0s
 => => unpacking to docker.io/library/myapp:1.0                                                                       1.7s
```

Docker 镜像构建成功，镜像名为 myapp:1.0

4.3 docker-compose 运行

```
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> docker compose -f docker/docker-compose.yml up -d --build
[+] up 10/10
 ✔Image redis:7-alpine Pulled                                                                                    21.5s
[+] Building 1.1s (12/12) FINISHED
 => [internal] load local bake definitions                                                                       0.0s
 => => reading from stdin 617B                                                                                   0.0s
 => [internal] load build definition from Dockerfile                                                             0.0s
 => => transferring dockerfile: 209B                                                                             0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                                              0.7s
 => [internal] load .dockerignore                                                                                0.0s
 => => transferring context: 2B                                                                                  0.0s
 => [1/5] FROM docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767af9  0.0s
 => => resolve docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767af9  0.0s
 => [internal] load build context                                                                                0.0s
 => => transferring context: 94B                                                                                 0.0s
 => CACHED [2/5] WORKDIR /app                                                                                    0.0s
 => CACHED [3/5] COPY src/requirements.txt .                                                                     0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                              0.0s
 => CACHED [5/5] COPY src/app.py .                                                                               0.0s
 => exporting to image                                                                                           0.1s
 => => exporting layers                                                                                          0.0s
 => => exporting manifest sha256:edb72503199948b402d668ac7172aaa9bd278d8ba41a2a03ed392ef5a9e8312c8                0.0s
 => => exporting config sha256:54675f2740e0677d775e9b10f43f871d8cc906364b6093a4ff4a16d3678548c7                  0.0s
 => => exporting attestation manifest sha256:acf47dee339d1a6f951c179cb32e59c2abf854c61b9df42c414a82d1b66d281a     0.0s
 => => exporting manifest list sha256:6283c1597bda23d2ecb8ccd503f6eb0f89e3fe951e31145a89d56d81abcb2466            0.0s
 => => naming to docker.io/library/docker-api:latest                                                             0.0s
 => => unpacking to docker.io/library/docker-api:latest                                                          0.0s
 => resolving provenance for metadata file                                                                       0.0s
[+] up 14/14
 ✔Image redis:7-alpine     Pulled                                                                                21.5s
 ✔Image docker-api         Built                                                                                  1.3s
 ✔Network docker_default   Created                                                                                0.0s
 ✔Container docker-redis-1 Started                                                                                0.4s
 ✔Container docker-api-1   Started                                                                                0.7s
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> Start-Sleep 10
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> docker compose -f docker/docker-compose.yml ps
NAME            IMAGE           COMMAND                  SERVICE   CREATED          STATUS          PORTS
docker-api-1    docker-api      "python app.py"          api       36 seconds ago   Up 35 seconds   0.0.0.0:8000->5000/tcp, [::]:8000->5000/tcp
docker-redis-1  redis:7-alpine  "docker-entrypoint.s…"   redis     36 seconds ago   Up 35 seconds   6379/tcp
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验>
```

compose 启动成功，api 与 redis 两个服务均为 running

4.4 容器访问验证

```
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> curl.exe http://localhost:8000/api
{"pod":"2b337f59259c","visits":1}
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> curl.exe http://localhost:8000/api
{"pod":"2b337f59259c","visits":2}
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> curl.exe http://localhost:8000/api
{"pod":"2b337f59259c","visits":3}
```

三次访问 visits 依次递增，说明 Redis 数据持久化生效

4.5 Docker 资源占用

```
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> docker stats --no-stream
CONTAINER ID   NAME            CPU %     MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I/O         PIDS
2b337f59259c   docker-api-1    0.01%     30.95MiB / 7.282GiB   0.41%     4.18kB / 2.93kB   30.4MB / 160kB    1
1ed68e238f0e   docker-redis-1  0.26%     5.395MiB / 7.282GiB   0.07%     3.07kB / 1kB      0B / 0B           6
```

docker stats 显示 api 与 redis 容器的 CPU 与内存占用

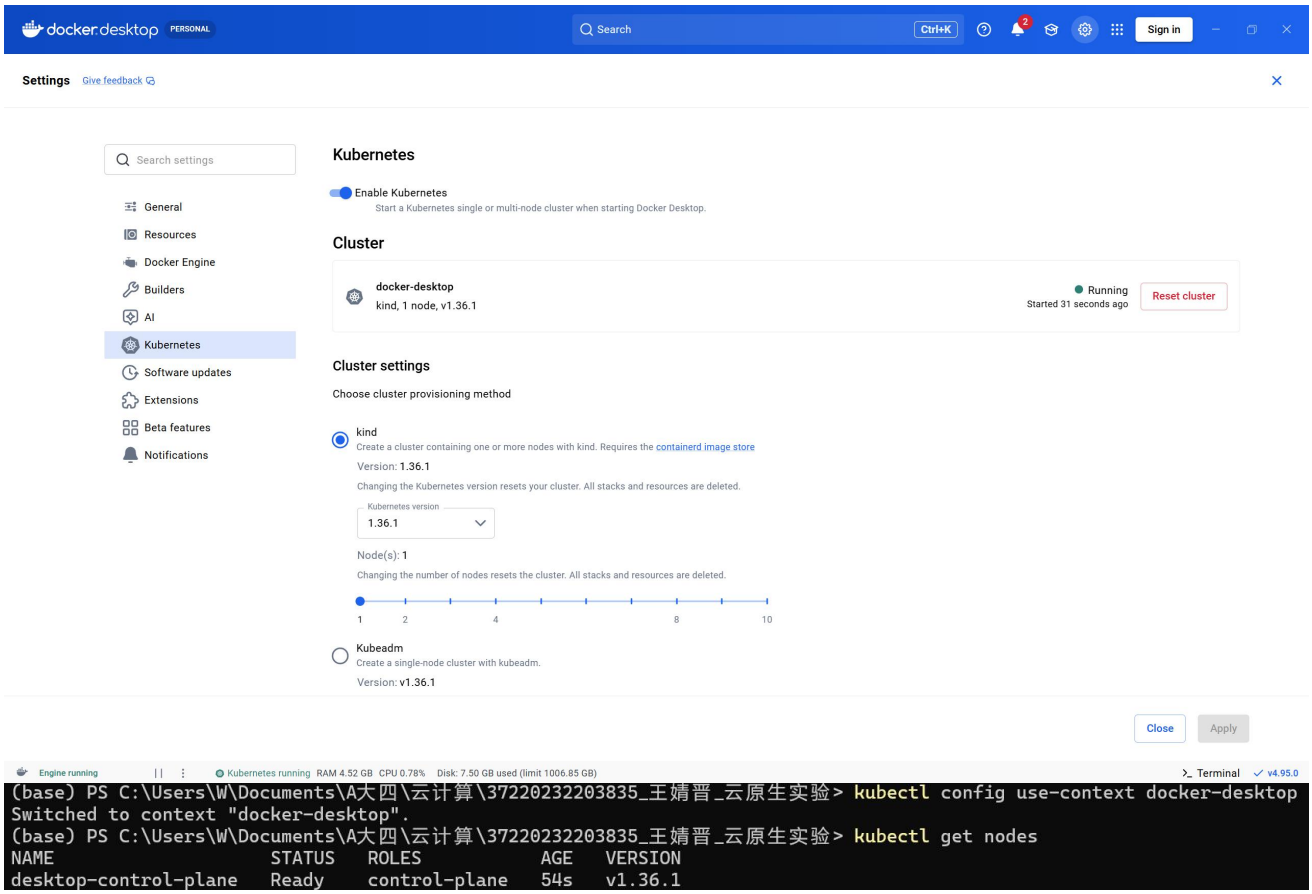
4.6 Minikube 集群启动

```
🏃  正在更新运行中的 docker "minikube" container ...
❗  从 Minikube 的 container 内部连接到 https://registry.k8s.io/ 失败
💡  要获取新的外部镜像，可能需要配置代理：https://minikube.sigs.k8s.io/docs/reference/networking/proxy/

❌  因 RUNTIME_ENABLE 错误而退出：Failed to enable container runtime: docker not running
```

图 6 Minikube 集群启动失败

实验过程中 Minikube 集群一直无法成功安装启动，使用原站与镜像地址均失败，本次实验直接使用 Docker Desktop 内置 Kubernetes



Kubernetes 集群启动成功，节点 desktop-control-plane 状态为 Ready

4.7 K8s 资源部署

```
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> kubectl config use-context docker-desktop
Switched to context "docker-desktop".
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> kubectl get nodes
NAME                  STATUS   ROLES           AGE   VERSION
desktop-control-plane   Ready    control-plane   54s   v1.36.1
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> cd "C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验"
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> kubectl apply -f k8s/all.yaml
configmap/app-config created
secret/app-secret created
deployment.apps/redis created
service/redis created
deployment.apps/api created
service/api created
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> Start-Sleep 30
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> kubectl get pods,svc
NAME                         READY   STATUS    RESTARTS   AGE
pod/api-5d64cb9494-h7wbz     1/1     Running   0          31s
pod/api-5d64cb9494-n695j     1/1     Running   0          31s
pod/redis-76599874fd-2wdnb   1/1     Running   0          31s

NAME                 TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
service/api          NodePort    10.96.118.247    <none>        5000:30080/TCP   31s
service/kubernetes   ClusterIP   10.96.0.1        <none>        443/TCP          3m26s
service/redis        ClusterIP   10.96.230.209    <none>        6379/TCP         31s
```

api 部署 2 副本、redis 部署 1 副本，均为 Running，Service 创建成功

4.8 K8s 服务访问验证

```
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> kubectl port-forward svc/api 8080:5000
Forwarding from 127.0.0.1:8080 -> 5000
Forwarding from [::1]:8080 -> 5000
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
```

kubectl port-forward 将本地 8080 端口转发到 api Service，处理了多次请求

```
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> curl.exe http://localhost:8080/api
{"pod":"api-5d64cb9494-h7wbz","visits":1}
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> curl.exe http://localhost:8080/api
{"pod":"api-5d64cb9494-h7wbz","visits":2}
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> curl.exe http://localhost:8080/api
{"pod":"api-5d64cb9494-h7wbz","visits":3}
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> curl.exe http://localhost:8080/api
{"pod":"api-5d64cb9494-h7wbz","visits":4}
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> curl.exe http://localhost:8080/api
{"pod":"api-5d64cb9494-h7wbz","visits":5}
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> curl.exe http://localhost:8080/api
{"pod":"api-5d64cb9494-h7wbz","visits":6}
```

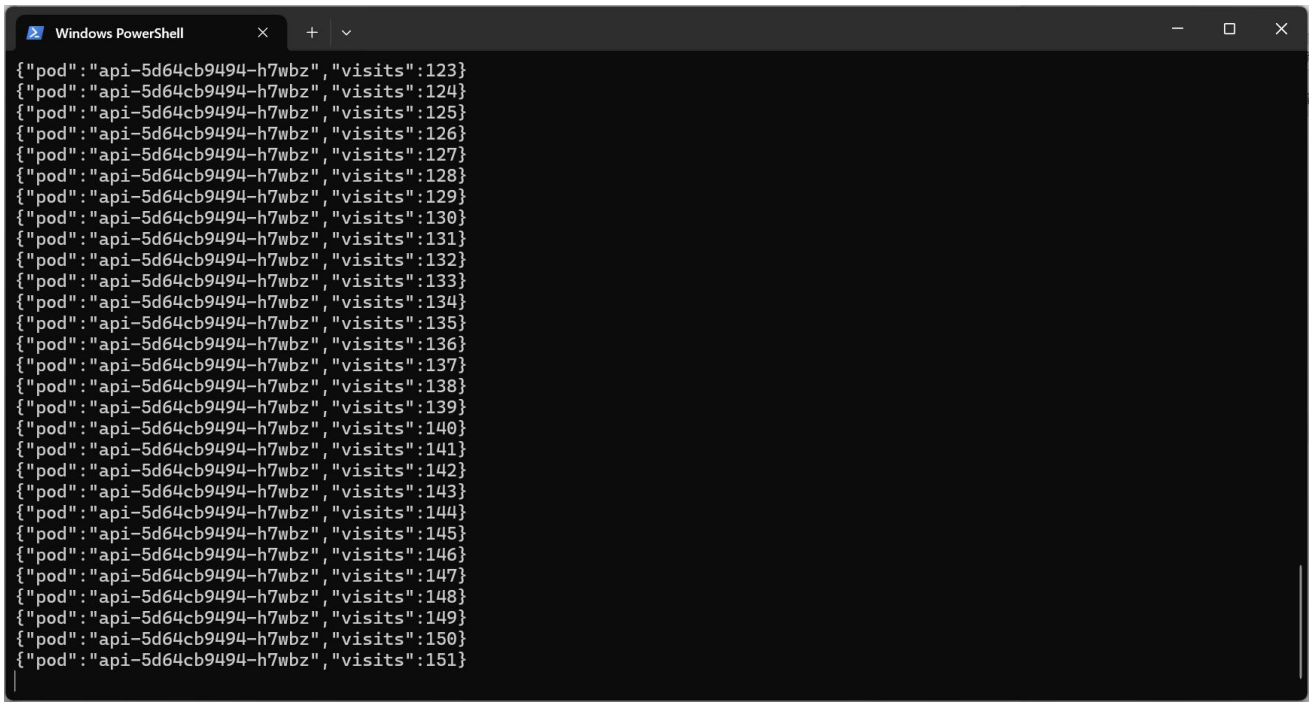
通过 Service 访问 api，visits 从 1 递增到 6，验证 Redis 数据持久化

4.9 自愈验证

持续访问的同时删除一个 api Pod:

```
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> kubectl get pods
NAME                     READY   STATUS    RESTARTS   AGE
api-5d64cb9494-h7wbz     1/1     Running   0          6m48s
api-5d64cb9494-n695j     1/1     Running   0          6m48s
redis-76599874fd-2wdnb   1/1     Running   0          6m48s
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> kubectl delete pod api-5d64cb9494-h7wbz
pod "api-5d64cb9494-h7wbz" deleted from default namespace
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> kubectl get pods -w
NAME                     READY   STATUS    RESTARTS   AGE
api-5d64cb9494-n695j     1/1     Running   0          8m40s
api-5d64cb9494-z9pv8     1/1     Running   0          35s
redis-76599874fd-2wdnb   1/1     Running   0          8m40s
redis-76599874fd-2wdnb   1/1     Running   0          8m56s
```

删除 Pod 后 K8s 自动重建新 Pod（z9pv8），api 副本数恢复为 2



删除 Pod 期间，持续访问未中断，visits 连续递增

4.10 扩缩容验证

```
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> kubectl scale deployment api --replicas=5
deployment.apps/api scaled
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> kubectl get pods
NAME                     READY   STATUS    RESTARTS   AGE
api-5d64cb9494-jjh7v     1/1     Running   0          1s
api-5d64cb9494-n695j     1/1     Running   0          12m
api-5d64cb9494-n7hlx     1/1     Running   0          1s
api-5d64cb9494-nvnhj     1/1     Running   0          1s
api-5d64cb9494-z9pv8     1/1     Running   0          4m38s
redis-76599874fd-2wdnb   1/1     Running   0          12m
```

扩容后 api 有 5 个 Pod

```
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> kubectl scale deployment api --replicas=2
deployment.apps/api scaled
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> kubectl get pods
NAME                     READY   STATUS        RESTARTS   AGE
api-5d64cb9494-jjh7v     1/1     Terminating   0          45s
api-5d64cb9494-n695j     1/1     Running       0          13m
api-5d64cb9494-n7hlx     1/1     Terminating   0          45s
api-5d64cb9494-nvnhj     1/1     Terminating   0          45s
api-5d64cb9494-z9pv8     1/1     Running       0          5m22s
redis-76599874fd-2wdnb   1/1     Running       0          13m
```

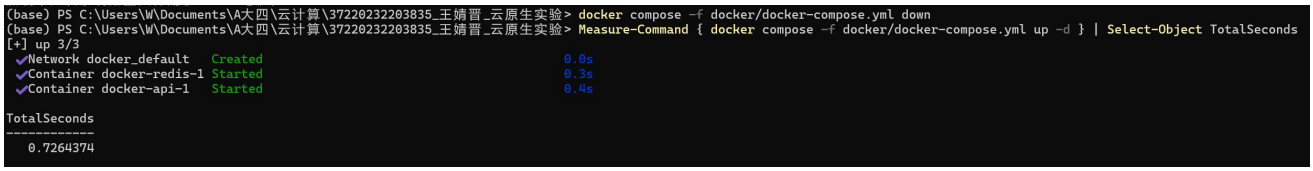
缩容后 api 恢复 2 个 Pod

```
(base) PS C:\Users\W\Documents\A大四\云计算\37220232203835_王婧晋_云原生实验> kubectl get pods,svc
NAME                         READY   STATUS    RESTARTS   AGE
pod/api-5d64cb9494-n695j     1/1     Running   0          14m
pod/api-5d64cb9494-z9pv8     1/1     Running   0          6m3s
pod/redis-76599874fd-2wdnb   1/1     Running   0          14m

NAME                 TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/api          NodePort    10.96.118.247   <none>        5000:30080/TCP   14m
service/kubernetes   ClusterIP   10.96.0.1       <none>        443/TCP          17m
service/redis        ClusterIP   10.96.230.209   <none>        6379/TCP         14m
```

最终状态：api 2 副本与 redis 1 副本均为 Running，Service 正常

五、消融对比实验



Docker Compose 启动两个容器耗时 0.73 秒



Kubernetes apply 耗时 0.20 秒，Pod 30 秒内全部 Running

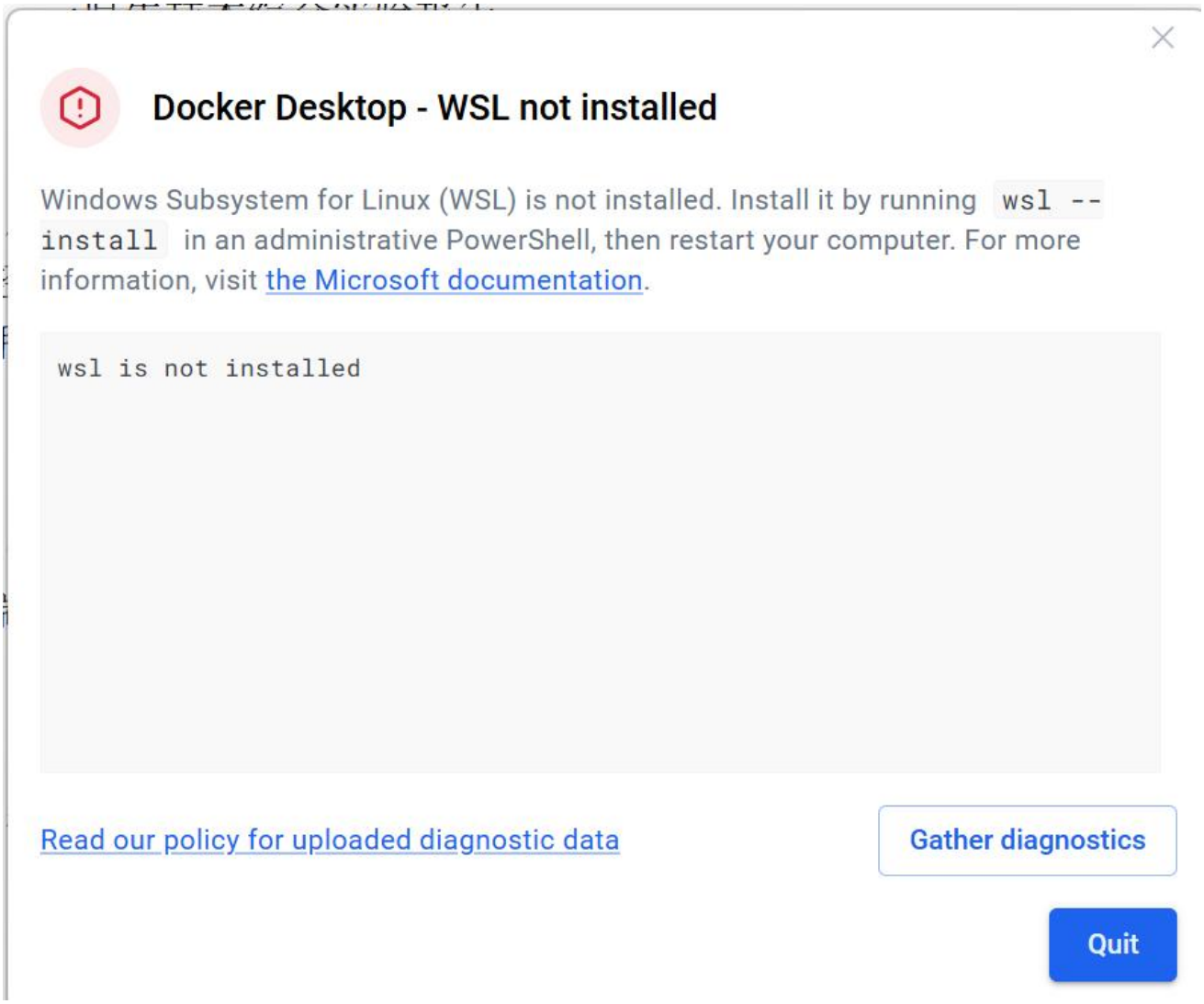


Docker 目录 2 个文件，K8s 单文件 65 行

结论：Docker 解决的是“应用在哪都能以一致方式运行”的问题，K8s 解决的是“应用大规模运行时如何自动调度、自愈、扩展”的问题。两者不是替代关系，而是叠加关系。本次实验中最直观的对比是：Docker 里删容器后需要手动重启，K8s 里删 Pod 后 35 秒内自动重建，且服务未中断。

六、问题与分析

Docker Desktop 报 WSL not installed:



首次打开 docker desktop 报错界面

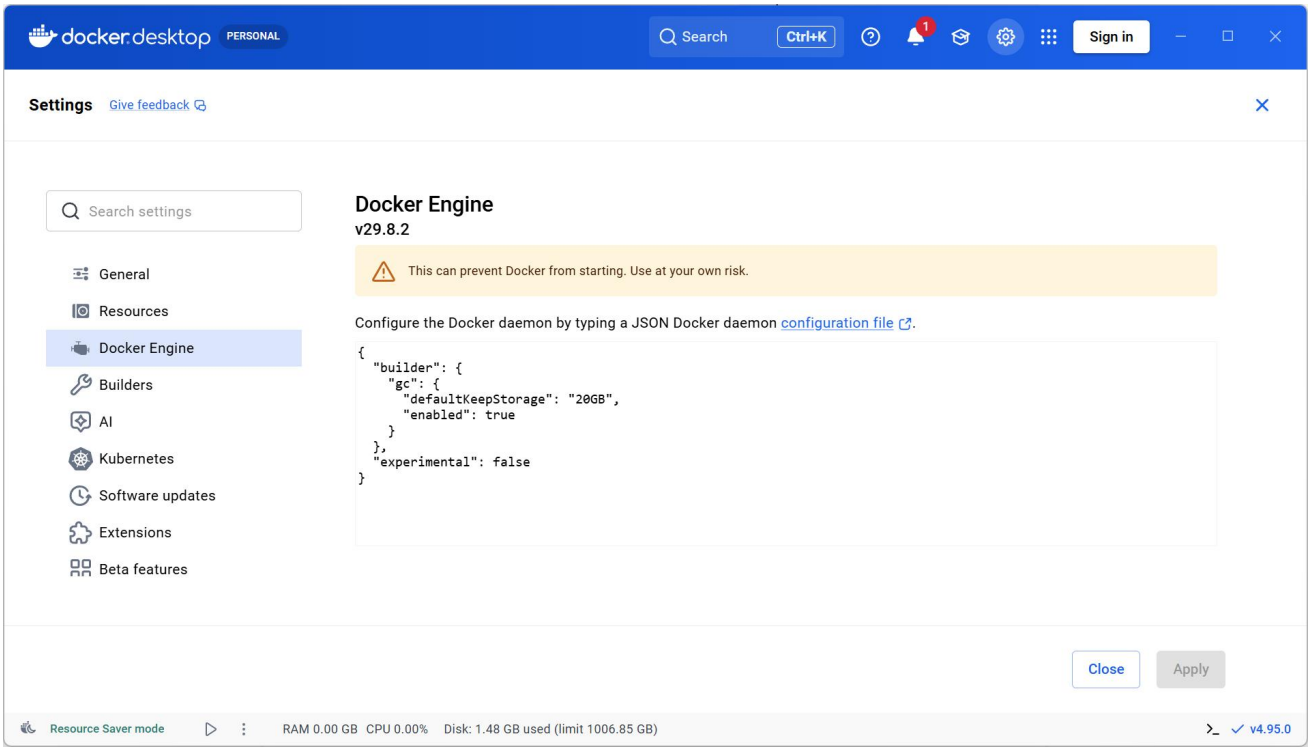
解决方法：以管理员身份运行 wsl --install，重启后执行 wsl --install -d Ubuntu 完成发行版安装。



WSL2 安装成功后的欢迎界面，Docker Desktop 集成已启用

docker build 拉取基础镜像超时:

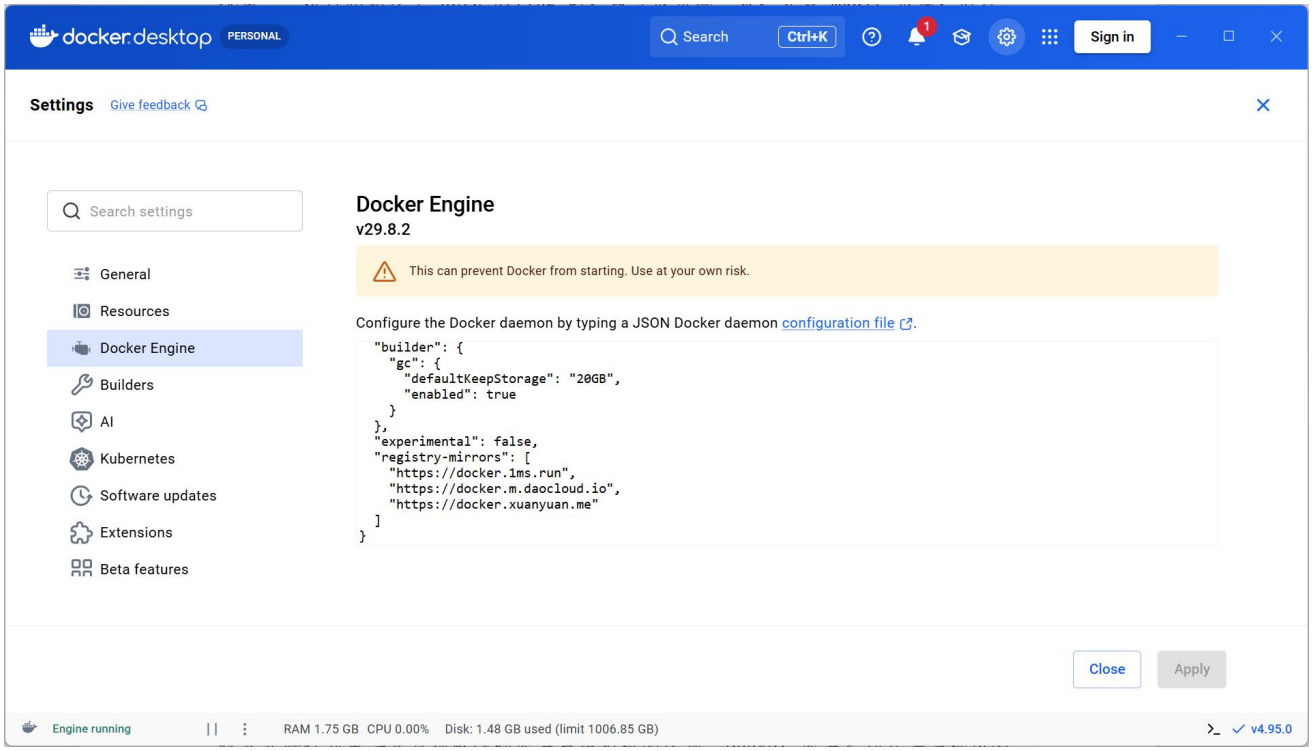
排查过程: docker info 显示 Registry Mirrors 为空。



配置前 Docker Engine 默认 JSON，registry-mirrors 为空

原因：Docker Hub 在国内网络环境下直连超时。

解决方法：Docker Desktop → Settings → Docker Engine，在 JSON 中增加 "registry-mirrors" 三个地址，Apply & Restart。



配置镜像加速器后，registry-mirrors 包含三个国内镜像源

复验结果：docker info 显示 Registry Mirrors 已生效，重新构建成功。

七、心得体会与技能收获

如果重做一次，我会一开始就选 Docker Desktop 内置的 Kubernetes，而不是在 Minikube 上反复折腾。我也会在实验开始前先配好 Docker 镜像加速器，而不是等到 docker build 一直超时无法进行下一步才去查。这两件事浪费了我三四个小时，下次我会早点开始准备实验，而不是等到最后来不及折腾虚拟机。

八、参考文献

Flask 官方文档：https://flask.palletsprojects.com

Redis 官方文档：https://redis.io/docs

Docker 官方文档：https://docs.docker.com

Kubernetes 官方文档：https://kubernetes.io/docs

Minikube 官方文档: https://minikube.sigs.k8s.io/docs

Kubernetes ConfigMap 与 Secret 使用指南:

https://kubernetes.io/docs/concepts/configuration/